

Kafka Session 1

=====

what is apache kafka?

Apache Kafka is an open-source distributed event streaming platform.

2 main things

=====

Data Integration / Collection

Data Processing - Kafka Streams

Socket - IP + port

Its not a replayable source

if we have a fast producer and we have a slow consumer...

producer is producing at 1000 records per second

consumer is consumit at 100 records per second..

Data Loss

we cannot have multiple consumers

By default the data can be retained for 7 days.

decouple the producer and consumer

iot devices

sensors -> Kafka -> Spark Structured Streaming -> Kakfa

pub-sub model

messaging system

Producer -> Kafka Cluster -> Consumer

Producer API

Consumer API

4 node kafka cluster

If you have to practise kafka

confluent.io

Managed Kafka on Cloud

\$400 free credits for a month.

producer
consumer
broker
cluster
topic
partitions
partition offset
consumer groups

Kafka Session 3

=====

Environment - default, dev, stage, prod

default -> cluster1, cluster2

Inside a Env you create clusters

Inside a cluster you would create a topic

environment -> cluster -> topic

Kafka Session 4

=====

(Key, Value)

```
{"order_id":1,"customer_id":11599,"customer_fname":"Mary","customer_lname":"Malone","city":"Hickory","state":"NC","pincode":28601,"line_items":[{"order_it
```

```
em_id":1,"order_item_product_id":957,"order_item_quantity":1,"order_item_pr  
oduct_price":299.98,"order_item_subtotal":299.98}}}
```

```
{  
  "order_id": 1,  
  "customer_id": 11599,  
  "customer_fname": "Mary",  
  "customer_lname": "Malone",  
  "city": "Hickory",  
  "state": "NC",  
  "pincode": 28601,  
  "line_items": [  
    {  
      "order_item_id": 1,  
      "order_item_product_id": 957,  
      "order_item_quantity": 1,  
      "order_item_product_price": 299.98,  
      "order_item_subtotal": 299.98  
    }  
  ]  
}
```

```
{  
  "order_id": 2,  
  "customer_id": 256,  
  "customer_fname": "David",  
  "customer_lname": "Rodriguez",  
  "city": "Chicago",  
  "state": "IL",  
  "pincode": 60625,  
  "line_items": [  
    {  
      "order_item_id": 2,  
      "order_item_product_id": 1073,  
      "order_item_quantity": 1,  
      "order_item_product_price": 199.99,  
      "order_item_subtotal": 199.99  
    },  
    {  
      "order_item_id": 3,  
      "order_item_product_id": 502,  
      "order_item_quantity": 5,  
      "order_item_product_price": 50,  
      "order_item_subtotal": 250  
    },  
  ]  
}
```

```
{
  "order_item_id": 4,
  "order_item_product_id": 403,
  "order_item_quantity": 1,
  "order_item_product_price": 129.99,
  "order_item_subtotal": 129.99
}
]
```

```
11599:{"order_id":1,"customer_id":11599,"customer_fname":"Mary","customer_lname":"Malone","city":"Hickory","state":"NC","pincode":28601,"line_items":[{"order_item_id":1,"order_item_product_id":957,"order_item_quantity":1,"order_item_product_price":299.98,"order_item_subtotal":299.98}]}
```

```
256:{"order_id":2,"customer_id":256,"customer_fname":"David","customer_lname":"Rodriguez","city":"Chicago","state":"IL","pincode":60625,"line_items":[{"order_item_id":2,"order_item_product_id":1073,"order_item_quantity":1,"order_item_product_price":199.99,"order_item_subtotal":199.99}, {"order_item_id":3,"order_item_product_id":502,"order_item_quantity":5,"order_item_product_price":50.0,"order_item_subtotal":250.0}, {"order_item_id":4,"order_item_product_id":403,"order_item_quantity":1,"order_item_product_price":129.99,"order_item_subtotal":129.99}]}
```

Kafka Session 5

=====

retail chain - walmart store

whenever we call produce() it puts the message in the producers pending buffer. It will asynchronously dispatch those messages. what it will then not do by itself is call the callback functions.

when you call poll(), the producer checks for the asynchronous events that needed to be processed. These events include ack from brokers for messages sent, error.

flush() should be called prior to shutting down the producer to ensure all outstanding/queued/in-flight messages are delivered.

```
from confluent_kafka import Producer
```

```

conf = {'bootstrap.servers': 'pkc-l7pr2.ap-south-1.aws.confluent.cloud:9092',
        'security.protocol': 'SASL_SSL',
        'sasl.mechanism': 'PLAIN',
        'sasl.username': 'Z2OSAATSA55GQVZK',
        'sasl.password':
'a1QC+319eHmlkhcb665BX6v2PCI5utizHlkPmZJQX43Tbx0ZtIWEht4888R1IJ
qq',
        'client.id': 'Sumit Macbook'}

```

```

producer = Producer(conf)

```

```

customer_id = "256"
customer_details =
'{"order_id":2,"customer_id":256,"customer_fname":"David","customer_lname"
:"Rodriguez","city":"Chicago","state":"IL","pincode":60625,"line_items":[{"order
_item_id":2,"order_item_product_id":1073,"order_item_quantity":1,"order_item
_product_price":199.99,"order_item_subtotal":199.99},{"order_item_id":3,"orde
r_item_product_id":502,"order_item_quantity":5,"order_item_product_price":5
0.0,"order_item_subtotal":250.0},{"order_item_id":4,"order_item_product_id":4
03,"order_item_quantity":1,"order_item_product_price":129.99,"order_item_su
btotal":129.99}]}'

```

```

def acked(err, msg):
    if err is not None:
        print(f'Failed to deliver message: {err}')
    else:
        msg_key = msg.key().decode('utf-8')
        msg_value = msg.value().decode('utf-8')
        print(f'Message produced key is: {msg_key} and value is: {msg_value}')

```

```

producer.produce("retail-data-new",key=customer_id,value=customer_details,
callback=acked)

```

```

producer.poll(1)

```

```

producer.flush()

```

```

=====

```

```

kafka session 6

```

```

=====

```

```

confluent-kafka[avro,json,protobuf]>=1.4.2

```

```

confluentBootstrapServers = 'pkc-l7pr2.ap-south-1.aws.confluent.cloud:9092'
confluentApiKey = 'Z2OSAATSA55GQVZK'

```

```
confluentSecret =  
'a1QC+319eHmlkhcb665BX6v2PCI5utizHlkPmZJQX43Tbx0ZtIWEht4888R1IJ  
qq'  
confluentTopicName = 'retail-data-new'
```

```
confluent-kafka[avro,json,protobuf]>=1.4.2
```

```
confluentBootstrapServers =  
confluentApiKey =  
confluentSecret =  
confluentTopicName =
```

```
kafka.bootstrap.servers  
kafka.security.protocol  
kafka.sasl.mechanism  
kafka.ssl.endpoint.identification.algorithm  
subscribe
```

JAAS - Java Authentication and Authorization Service

```
.option("kafka.sasl.jaas.config",  
"kafkashaded.org.apache.kafka.common.security.plain.PlainLoginModule  
required username='{}' password='{}';".format(confluentApiKey,  
confluentSecret))
```

```
orders_schema = "order_id long,customer_id long,customer_fname  
string,customer_lname string,city string,state string,pincode long,line_items  
array<struct<order_item_id: long,order_item_product_id:  
long,order_item_quantity: long,order_item_product_price:  
float,order_item_subtotal: float>>"
```

```
source -> kafka
```

```
target -> delta
```

kafka session 9

=====

kafka producer using pyspark

read the data from kafka topic -> write the results to bronze (persistent storage) -> processing -> write to a kafka topic

Python producer -> Pyspark kafka consumer -> Persistent storage -> Processing -> produce using pyspark kafka

where city = 'Chicago'

